

OS Development

Coroutines and Threads

Prof. Dr. Michael Schöttner
Dr. Fabian Ruhland

- Motivation

- Virtual Parallelism

- Coroutines

- Threads

- Summary

- We want to execute functions in parallel (asynchronous)
- Example: Receive network message, or read data from a hard disk

- Motivation

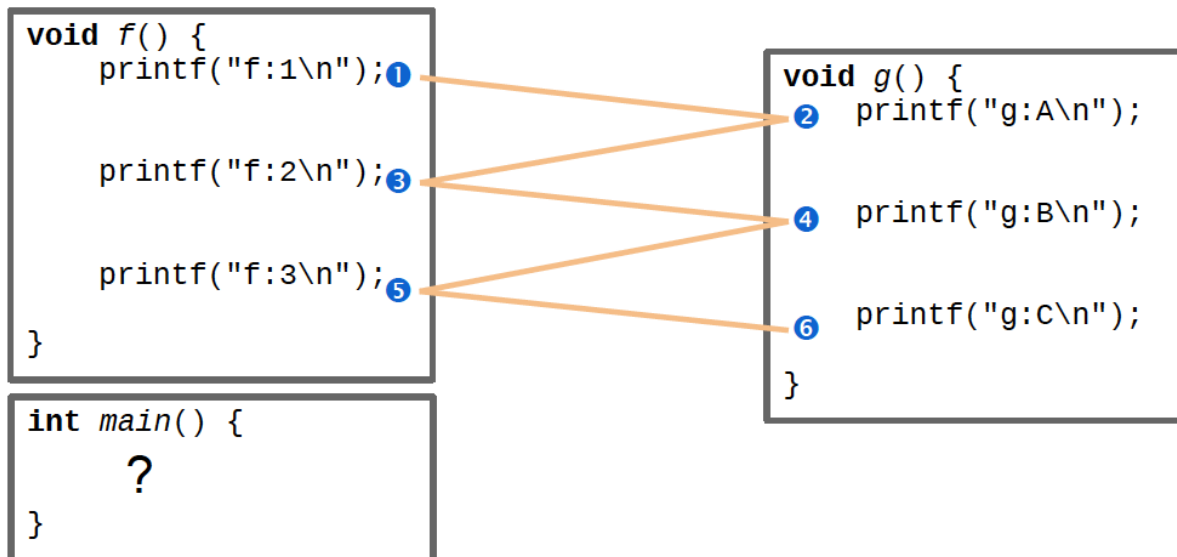
- Virtual Parallelism

- Coroutines

- Threads

- Summary

- Given: Functions f and g
- Goal: f and g should run *interlaced*



Virtual Parallelism: Experiment 1

```
void f() {  
    printf("f:1\n");  
  
    printf("f:2\n");  
  
    printf("f:3\n");  
}
```

```
int main() {  
    f();  
    g();  
}
```

```
void g() {  
    printf("g:A\n");  
  
    printf("g:B\n");  
  
    printf("g:C\n");  
}
```

```
os@ios>gcc routine.c -o routine  
os@ios>./routine  
f:1  
f:2  
f:3  
g:A  
g:B  
g:C
```

Obviously does not
work...

Virtual Parallelism: Experiment 2

```
void f() {  
    printf("f:1\n");  
    g();  
  
    printf("f:2\n");  
    g();  
  
    printf("f:3\n");  
    g();  
}
```

```
int main() {  
    f();  
}
```

```
void g() {  
    printf("g:A\n");  
  
    printf("g:B\n");  
  
    printf("g:C\n");  
}
```

```
os@ios>gcc routine.c -o routine  
os@ios>./routine  
f:1  
g:A  
g:B  
g:C  
f:2  
...
```

Does not work either...

Virtual Parallelism: Experiment 3

```
void f() {  
    printf("f:1\n");  
    g();  
  
    printf("f:2\n");  
    g();  
  
    printf("f:3\n");  
    g();  
}
```

```
int main() {  
    f();  
}
```

Does not work at all!

```
void g() {  
    printf("g:A\n");  
    f();  
  
    printf("g:B\n");  
    f();  
  
    printf("g:C\n");  
    f();  
}
```

```
os@ios>gcc routine.c -o routine  
os@ios>./routine  
f:1  
g:A  
f:1  
g:A  
...  
Segmentation fault
```


Virtual Parallelism: Experiment 4

```
void f_start() {  
    printf("f:1\n");  
    f = &l1; goto *g;  
  
l1: printf("f:2\n");  
    f = &l2; goto *g;  
  
l2: printf("f:3\n");  
    goto *g;  
}
```

```
void (*volatile f)();  
void (*volatile g)();  
  
int main() {  
    f = f_start;  
    g = g_start;  
    f();  
}
```

This works!

```
void g_start() {  
    printf("g:A\n");  
    g = &l1; goto *f;  
  
l1: printf("g:B\n");  
    g = &l2; goto *f;  
  
l2: printf("g:C\n");  
    exit(0);  
}
```

```
os@ios>gcc-2.95 -o coroutine coroutine.c  
os@ios>./coroutine  
f:1  
g:A  
f:2  
g:B  
f:3  
g:C
```

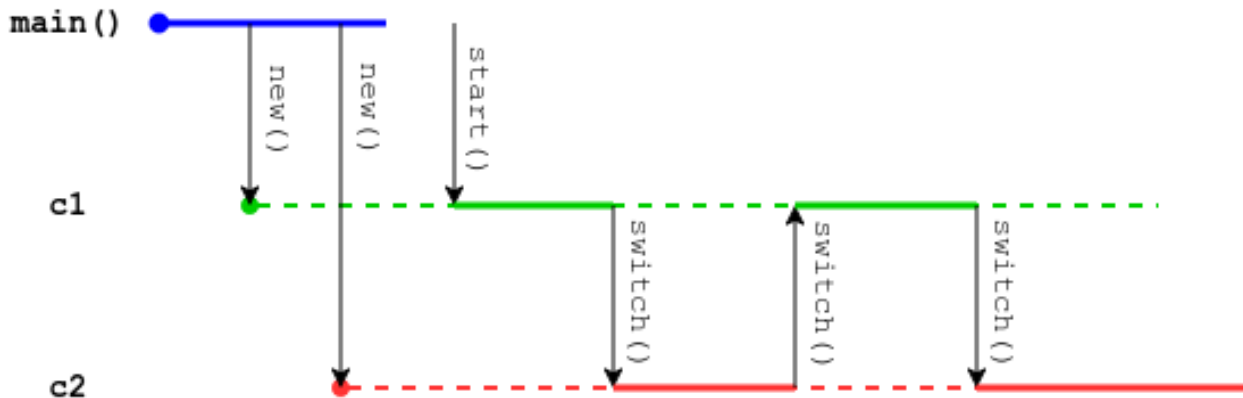
- Virtual parallelism cannot be achieved using function calls
 - Simple function calls (experiment 1 and 2) → Always execute the function fully
 - Recursive function calls (experiment 3) → Same, leading to endless recursion and stack overflow
- Functions must be able to be exited and continued during execution
 - Somewhat like in experiment 4...
 - Instruction pointer stored in global variables and resumed using `goto`
 - ...but without the involved problems
 - Direct jumps out of and into functions are undefined behaviour in C! (`goto` with pointers is a GCC feature)
 - Function state does not only consist of the instruction pointer – What about CPU registers and stack?

- Motivation
- Virtual Parallelism
- Coroutines
- Threads
- Summary

- **Coroutine:** Function that can be interrupted and continued at a later point in time
- Each coroutine has its own register set and stack
- Similar to a thread, but not known to scheduler
- The concept of coroutines comes from programming languages
 - E.g., Kotlin, Rust, Python, Simula-67, ...

- Coroutines are executed in sequence
 - State of each coroutine is conserved across entry/exit
- All coroutines have equal rights
 - Order of execution is determined by the coroutines themselves
 - Either static (HeineOS)
 - Or dynamic (active coroutine decides which one is next)
 - Order of execution is programmed, contrary to a dynamic schedule (determined by a scheduler)

- We implement a static schedule → Coroutines are linked in a cycle
- Each coroutines simply calls `switch()` to switch to next coroutine



- We implement coroutines as a preliminary stage for cooperative/preemptive threads

- Motivation
- Virtual Parallelism
- Coroutines
- Threads
- Summary

- *A thread is a coroutine, which is known to the scheduler*
- Scheduler manages threads and decides, when each thread may run (see next chapter)
- Each thread belongs to a process (address space)
 - HeineOS only has one process (Kernel) → Only one address
- Concept origins:
 - Multithreading is historically (rather) an operating system feature
 - Coroutines are historically (rather) a programming language feature

- Some programming languages support coroutines through the compiler, combined with runtime function to enable asynchronous programming
- Primary use: Blocking I/O
 - When a call blocks, the CPU is implicitly handed off
 - An executor is responsible for switching to the next coroutine
 - Coroutines are not statically linked in this case
- Rust: `async & await`
- Java: Project Loom

- Motivation
- Virtual Parallelism
- Coroutines
- Threads
- Summary

- Coroutines are functions, that can be interrupted in their execution
 - Similar to a thread, but not known to the operating system
- Threads are coroutines, that are managed by the operating system's scheduler
- We implement coroutines as a preliminary stage to our cooperative/preemptive threads